

Universidad Rafael Landívar

Facultad de Ingeniería

Ingeniería en Informática Y Sistemas

Lenguajes Formales Y Automatas

Catedrático: Ing. Damaris Campos

Proyecto NO.2 (Fase 2)

“Manual Técnico”

Pedro Pablo Cossio Rivas-1054723

Rodrigo José Ruiz Juarez-1037623

Manual Técnico

Analizador de Operaciones Aritméticas

1. Introducción Técnica

Propósito del Documento

Este manual técnico describe la implementación del Analizador de Operaciones Aritméticas, proporcionando detalles sobre la arquitectura, algoritmos y estructuras de datos utilizadas. Está dirigido a desarrolladores y personas con conocimientos técnicos que necesiten entender o modificar el sistema.

Metodología de Desarrollo

El proyecto se desarrolló siguiendo el método del árbol para la construcción del analizador léxico y sintáctico. Se diseñó una estructura modular con las siguientes etapas principales:

- Definición de expresiones regulares para el léxico.
- Construcción del árbol sintáctico abstracto (AST).
- Implementación de evaluación mediante patrón Visitor.
- Generación de reportes en formato HTML.
- Integración de interfaz gráfica Tkinter para la interacción del usuario.

Tecnologías Utilizadas

Componente	Tecnología	Versión
Lenguaje de Programación	Python	3.7+
Interfaz Gráfica	Tkinter	Built-in
Tipado	typing	Built-in
Operaciones Matemáticas	math	Built-in

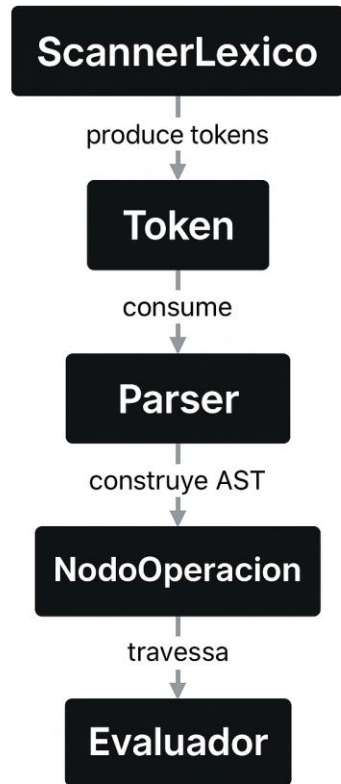
2. Arquitectura del Sistema

El sistema sigue una arquitectura de pipeline con cuatro fases principales de compilación:



3. Diagrama de Clases

Diagrama de Clases



- **Token**
Guarda la información básica de cada elemento reconocido por el analizador léxico: tipo, texto (lexema), fila y columna.
- **ScannerLexico**
Lee el texto de entrada y lo convierte en una lista de *tokens*. Detecta y registra errores léxicos.
- **NodoOperacion**
Representa un nodo dentro del árbol de operaciones (AST). Cada nodo puede ser una operación (SUMA, RESTA, etc.) o un número.
- **Parser**
Toma los tokens del léxico y construye el árbol sintáctico (AST). Verifica que la estructura del código sea válida y jerárquica.
- **Evaluador**
Recorre el árbol (AST) y calcula el resultado numérico de las operaciones. Maneja errores como divisiones entre cero o raíces inválidas.

- **Exportador HTML**
Genera los archivos Resultados.html y Errores.html mostrando los resultados y errores encontrados.
- **Interfaz Gráfica (Tkinter)**
Permite al usuario abrir, editar, analizar y exportar resultados mediante una interfaz visual.

4. Análisis Léxico

El analizador léxico transforma el texto de entrada en una lista de tokens significativos (números, etiquetas, operadores).

Cada token contiene su tipo, lexema y posición (fila, columna). Los errores léxicos se registran cuando hay caracteres no válidos.

Token	Expresión Regular
NUMERO	<code>[0-9]+(\.[0-9]+)?</code>
OPERACION	<code><Operacion=[A-Za-z]+></code>
ETIQUETA_CIERRE	<code></[A-Za-z]+></code>
P	<code><P></code>
R	<code><R></code>

ETIQUETA_ABIERTA	<Operacion= SUMA>	Fila:1 Col:1
ETIQUETA_ABIERTA	<Numero>	Fila:2 Col:5
NUMERO	3.2	Fila:2 Col:14
ETIQUETA_CERRADA	</Numero>	Fila:2 Col:18
ETIQUETA_ABIERTA	<Numero>	Fila:3 Col:5
NUMERO	7.8	Fila:3 Col:14
ETIQUETA_CERRADA	</Numero>	Fila:3 Col:18
ETIQUETA_ABIERTA	<Numero>	Fila:4 Col:5
NUMERO	1	Fila:4 Col:14
ETIQUETA_CERRADA	</Numero>	Fila:4 Col:18
ETIQUETA_CERRADA	</Operacion>	Fila:5 Col:1
ETIQUETA_ABIERTA	<Operacion= RESTA>	Fila:7 Col:1
ETIQUETA_ABIERTA	<Numero>	Fila:8 Col:5
NUMERO	50	Fila:8 Col:14
ETIQUETA_CERRADA	</Numero>	Fila:8 Col:18
ETIQUETA_ABIERTA	<Numero>	Fila:9 Col:5
NUMERO	12.75	Fila:9 Col:14
ETIQUETA_CERRADA	</Numero>	Fila:9 Col:20

5. Análisis Sintáctico

El parser toma la lista de tokens y construye un Árbol Sintáctico Abstracto (AST). Cada nodo del árbol representa una operación, número o parámetro dentro de la estructura jerárquica.

GRAMÁTICA BASE:

programa -> (operacion | numero)*

operacion -> <Operacion=TIPO> contenido </Operacion>

contenido -> (<Numero> NUM </Numero> | <P> NUM </P> | <R> NUM </R> | operacion)*

Analizador - Entrada1.txt
rchivo Ayuda

Abrir Guardar Guardar Como Analizar Exportar HTML Manual de Usuario (PDF) Manual Técnico (PDF)

Código Fuente:

```
</Operacion>
<Operacion= MULTIPLICACION>
  <Numero> 6 </Numero>
  <Numero> 2.5 </Numero>
  <Numero> 3 </Numero>
</Operacion>
<Operacion= DIVISION>
  <Numero> 81 </Numero>
  <Numero> 3 </Numero>
</Operacion>
<Operacion= POTENCIA>
  <P> 3 </P>
  <Operacion= SUMA>
    <Numero> 1.5 </Numero>
    <Numero> 2.5 </Numero>
  </Operacion>
</Operacion>
<Operacion= RAIZ>
  <R> 5 </R>
  <Numero> 243 </Numero>
</Operacion>
<Operacion= INVERSO>
  <Operacion= MULTIPLICACION>
    <Numero> 4 </Numero>
    <Numero> 5 </Numero>
  </Operacion>
</Operacion>
<Operacion= MOD>
  <Numero> 58 </Numero>
  <Numero> 9 </Numero>
</Operacion>
<Operacion= SUMA>
  <Numero> 5 </Numero>
  <Operacion= POTENCIA>
    <P> 2 </P>
    <Operacion= RAIZ>
      <R> 3 </R>
      <Numero> 125 </Numero>
    </Operacion>
  </Operacion>
</Operacion>
```

Resultados:
Tokens Árbol Expresión y Resultado Errores

Mostrando operación 5 de 9

```
graph TD
  A[A] --> B[3.00]
  A --> C[+]
  C --> D[1.50]
  C --> E[2.50]
```

6. Evaluación de Expresiones

El evaluador recorre el AST aplicando operaciones matemáticas. Soporta SUMA, RESTA, MULTIPLICACION, DIVISION, POTENCIA, RAIZ, INVERSO y MOD.

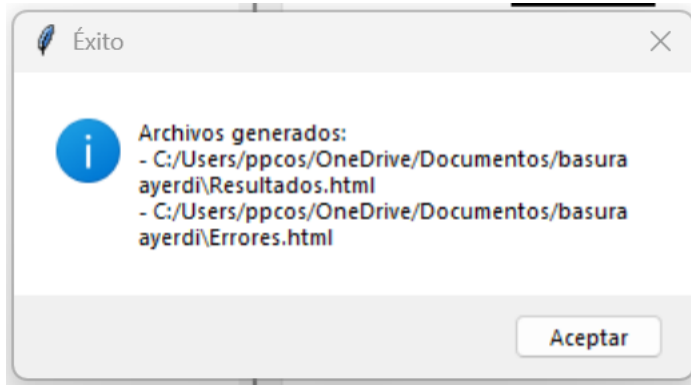
Cada operación se valida antes de su ejecución para evitar errores como división por cero o dominio inválido.

Operación	Algoritmo	Validación
SUMA	sum(valores)	≥ 2 operandos
RESTA	valores[0] - valores[1] ...	≥ 2 operandos
DIVISION	valores[0] / valores[1]	divisor $\neq 0$
POTENCIA	base ** exponente	2 operandos exactos
RAIZ	radicando ** (1/indice)	indice $\neq 0$
INVERSO	1 / valor	valor $\neq 0$
MOD	a % b	b $\neq 0$

7. Generación de Reportes

El sistema genera dos archivos HTML:

- Resultados.html: muestra las expresiones procesadas con su resultado.
- Errores.html: combina errores léxicos y semánticos con su ubicación (fila, columna).



8. Diagrama de Flujo General

Flujo General

